# A Bit Shift Wearing A Disguise

**The Collatz map as the one-sided binary shift: a code paper, with computed checks**

**Cody Michael Allison** (Michael Rendier), independent researcher · the.wandering.god@gmail.com · ORCID 0009-0007-7239-6760 · github.com/michaelrendier
**Written by an AI assistant:** Claude Sonnet 5.5 (`claude-sonnet-5-5`), with the author's `cs-paper-code-conventions` skill running
**Date:** 14 October 2026 · **Status:** draft for expert review · **Code:** FourthAgePapers/CollatzShift

---

## Read this first: who wrote this, and how

**The assistant.** This paper was written by a transformer language model, Claude Sonnet 5.5 (`claude-sonnet-5-5`), working with the author's `cs-paper-code-conventions` skill active; the skill decides how code is shown in a paper (it is linked above and described in Appendix A). The original engine and README for this topic were committed on 2026-08-21 co-authored by an earlier model (Claude Opus 5); this rewrite, the new code in `engine/`, and the text are from 2026-10-14.

**The author.** The author is a software engineer, not a trained mathematician, and says so. He is asking mathematicians to read this critically. **Every statement here that is not attributed to a cited source should be treated as unverified until an expert has looked at it.**

**The method.** The author deliberately does not tell the assistant what result he is looking for or hoping for. Tasks are given as measurements ("run it through the engine", "chase that anomaly to its source"), so that he is not handed the answers he wants. This is a safeguard, not a guarantee: an assistant can still infer an expectation from context. The real safeguards are exact enumeration, a control before each measurement, and the rule that **failures stay in the code and in the paper** (Section 7).

**What kind of paper this is.** A code paper. It shows the programs, what they compute, how to run them, and what came out. It contains no proofs. Where a result is in the literature it says so and cites it.

---

## Abstract

We describe three small, self-checking Python programs (standard library only, a few seconds each) and report what they compute about the Collatz shortcut map T(n) = n/2 (n even), T(n) = (3n+1)/2 (n odd). The first exhibits T, on residue classes mod 2^k, as the one-sided binary shift under the parity-vector conjugacy of Lagarias, Bernstein and Bernstein–Lagarias, and re-checks ten relations exhaustively or by stated sampling. The second re-computes the numerical tables of Bernstein and Lagarias (1996) for the conjugacy map: the cycle table agrees for every row we computed (n ≤ 16), the order 2^(n−4) and the power-of-two cycle lengths are reproduced, and the counts of odd fixed points agree with 86 of 99 entries of their Table 2.3 and 99 of 100 entries of their Table 2.4 for n up to 1000. The disagreements form one block (n = 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78 in Table 2.3; n = 70 in Table 2.4) in which the printed number is our number for n+1; we cannot tell whether the printed table or our reading of it is at fault, and we ask readers to adjudicate. The third reads the map through the author's GenerationalLineage engine, which decomposes operations into three primitives, ADD, SCALE and SIGN; we find that the engine's SIGN (a reflection) occurs

in neither branch of T, so the earlier statement "T = ADD ∘ SCALE ∘ SIGN" in this repository conflated a parity selector with that primitive, and we correct it. A control that appeared to show a statistical signature in Collatz parity streams is traced to its source and found to be an artefact of our own matching. We make no claim about the conjecture.

---

## 0. Scope and labels

**Not a Fourth Age paper.** This directory sits in the `FourthAgePapers` repository for historical reasons. The author's Fourth Age template ("One Paper, One Claim") asks for a pre-registered prediction. This paper is a results-and-code paper: it makes **no prediction**, nothing in it is pre-registered, and it is not an instance of that template.

**Labels.** Each result carries a label.

| label | meaning |
|---|---|
| `ESTABLISHED` | stated in the literature (cited); here only re-computed |
| `OURS` | the author's reading or measurement; it may well be known, we did not find it stated |
| `THEORETICAL` | an interpretation inside the author's framework, offered for review |
| `OPEN` | a question we cannot settle |
| suffix `:CALCULATED` | the number or statement is produced by code in this repository and reproduced in this paper (`python3 engine/...`); **CALCULATED is not DERIVED**: an engine ran, deterministically and inspectably, but no symbolic proof chain is claimed |

A label is placed next to the result it describes. A result without code carries no `:CALCULATED`. The original engine prints its own provenance word, `KNOWN` or `OURS`; this paper maps `KNOWN` to `ESTABLISHED` and appends `:CALCULATED`, and the engine's blocks are shown verbatim, so both words appear.

**Provenance of the literature: built-against and Post-Hoc.** The project distinguishes work the author built against from work he found afterwards. **The Collatz conjecture is the one thing here the author engineered against** (built-against, no tag): it was the first test of the GenerationalLineage engine, chosen as a simple ordering of the integers before any of the literature below had been read. **Everything else cited in this paper was noticed after the code already ran and is marked `Post-Hoc:`** in the references. In particular, the author did not know the Bernstein–Lagarias results when the original engine was written; that his engine reproduces their tables is a check on the code, not evidence that he derived them.

## 1. Introduction

The Collatz conjecture asserts that every positive integer reaches 1 under iteration of T. It is well known that T, extended to the 2-adic integers $Z_2$, is conjugate to the shift (Lagarias 1985; Bernstein 1994; Bernstein–Lagarias 1996), and that the difficulty therefore sits in how the integers lie inside $Z_2$ and not in the dynamics of the shift itself. This paper does not add to that. It does three narrower things: it shows, with runnable code, ten facts about T that follow from the conjugacy and from elementary counting; it re-computes the published numerical tables of Bernstein–Lagarias and records exactly where our numbers and theirs differ; and it

reads T through one particular software framework, the GenerationalLineage engine, whose first real test this map was in August 2026.

**What we do not claim.** Nothing here proves or disproves the conjecture, and nothing here is evidence for or against it.

## 2. Definitions and the literature

Let T be as above. For x in $\mathbb{Z}_2$ let Q(x) = Σ_{i≥0} (T^i(x) mod 2) 2^i be its **parity vector**, and Q_n(x) = Q(x) mod 2^n. Let S be the shift S(x) = ⌊x/2⌋. Then Q(T(x)) = S(Q(x)), so Q conjugates T to S; Φ = $Q^{-1}$ is the conjugacy map of Bernstein–Lagarias, normalised by Φ(0) = 0. Q_n is a permutation of Z/2^n (a "solenoidal" bijection), and the Collatz conjecture is equivalent to the statement that every positive integer lies in Φ(−⅓Z) (Bernstein–Lagarias, Section 1). **[ESTABLISHED]** (Terras 1976; Lagarias 1985; Bernstein 1994; Bernstein–Lagarias 1996.) We use their definitions throughout; the copies we read are in `references/`.

## 3. The code

Three engines, each standard-library Python, each returning a dictionary and printing a transcript:

| file | what it does | time here |
|---|---|---|
| `engine/e_collatz_shift.py` | the original engine (2026-08-21, unchanged): ten relations about T, each with a provenance label | about 4 s |
| `engine/e_collatz_bl_replication.py` | re-computes the Bernstein–Lagarias tables with two independent algorithms | about 3 s |
| `engine/e_collatz_gl_decomposition.py` | reads T through GenerationalLineage; the control that went wrong | about 7 s |

`python3 paper/make_paper.py` rebuilds this paper from them: every number from code below is taken from a fresh run (the two sentences in Section 5.9 that say "in an earlier run" refer to exploratory scripts kept in ContextPlease, see Appendix A), every listing is extracted verbatim from the engine source and executed in an empty namespace before it is printed, and `--check` reports whether the committed text still matches a fresh build. The files' SHA-256 prefixes are in Section 9.

**Listing 1: the map, the parity vector, and the affine form of T^k on a residue class** (`engine/e_collatz_shift.py`, extracted verbatim; one complete unit)

```
from __future__ import annotations
from fractions import Fraction
from typing import Tuple


def T(n: int) -> int:
    """The shortcut Collatz map. SIGN selects, SCALE acts, ADD couples."""
    return n // 2 if n % 2 == 0 else (3 * n + 1) // 2
```

```python
def parity_vector(n: int, k: int) -> int:
    """parity_word packed little-endian into an integer of Z/2^k."""
    v, m = 0, n
    for i in range(k):
        v |= (m & 1) << i
        m = T(m)
    return v


def branch_affine(k: int, j: int) -> Tuple[int, Fraction, Fraction]:
    """For n = j (mod 2^k):  T^k(n) = (3^d n + c)/2^k.  Returns (d, slope, c).

    Fits the affine map from two lifts and then CHECKS the slope against
    3^d/2^k rather than assuming it — a fit that is never tested is a code
    fault waiting to happen.
    """
    lifts = []
    for t in range(3):
        n = j + t * (1 << k)
        m, d = n, 0
        for _ in range(k):
            if m & 1:
                d += 1
            m = T(m)
        lifts.append((n, m, d))
    ds = {L[2] for L in lifts}
    if len(ds) != 1:
        raise ValueError(f'd not constant on class {j} mod 2^{k}: {ds}')
    d = ds.pop()
    (n0, m0, _), (n1, m1, _) = lifts[0], lifts[1]
    slope = Fraction(m1 - m0, n1 - n0)
    c = Fraction(m0) - slope * n0
    return d, slope, c
```

Cold run of Listing 1:

```
T(7), T(8) = 11 4
parity_vector(7, 4) = 7 (binary 1110, low bit first)
class 7 mod 16: d = 3  slope = 27/16  c = 19/16  i.e. T^4(n) = (27 n + 19)/16
```

The last line is the whole content of "T is affine on classes": for $n \equiv 7 \pmod{16}$, four steps of T are $n \mapsto (27n + 19)/16$ (check: $n = 7$ gives 13, and $7 \to 11 \to 17 \to 26 \to 13$), and the exponent $d = 3$ counts the odd steps, which the class alone determines. (The docstring of `T` in this listing, "SIGN selects, SCALE acts, ADD couples", is the origin of the labelling that Section 5.7 corrects.)

**Listing 2: the odd fixed points of Q_n, by an independent search** (`engine/e_collatz_bl_replication.py`)

```python
def odd_fixed_points_dfs(N: int) -> dict[int, int]:
    """A second, independent algorithm for the same counts (n up to 1000 in about 2 s).

    For a fixed point x of Q_n the parity of T^j(x) must equal bit j of x. Along the parity word
    (which is the low bits of x)
    T^j(x) = (3^d x + c) / 2^j, and the parity of T^j(x) equals f_j XOR x_j, where f_j = ((3^d
    x_low + c) / 2^j) mod 2 depends only on
```

```
    the bits of x below j. So bit j is free when f_j = 0 and the branch dies when f_j = 1. Counts
↪   the surviving residues at each n."""
    p3 = [1]
    nodes = [(1, 1, 1)]                     # x_0 = 1 (odd): x_low = 1, d = 1 odd step, c = 1
    counts = {1: 1}
    for j in range(1, N):
        nxt = []
        for x_low, d, c in nodes:
            while len(p3) <= d:
                p3.append(p3[-1] * 3)
            if ((p3[d] * x_low + c) >> j) & 1:
                continue
            nxt.append((x_low, d, c))                                       # bit j = 0: even
↪   step, c unchanged
            nxt.append((x_low | (1 << j), d + 1, 3 * c + (1 << j)))         # bit j = 1: odd step
        nodes = nxt
        counts[j + 1] = len(nodes)
    return counts
```

```
{2: 2, 5: 6, 10: 10, 15: 18, 20: 26}
```

Bit j of a fixed point is free exactly when a quantity f_j, computable from the lower bits, is 0; otherwise the branch dies. This is a second algorithm for the same counts as the “extend one bit at a time” method in the same file, and the two agree (Section 5.8).

**Listing 3: parity stream of an orbit, and the number of Lyndon factors** (`engine/e_collatz_gl_decomposition.` used in Section 5.9)

```
def T(n): return n // 2 if n % 2 == 0 else (3 * n + 1) // 2   # as in Listing 1


def duval_count(s: str) -> int:
    """Number of factors in the Lyndon factorisation of s (Duval's algorithm)."""
    n, i, c = len(s), 0, 0
    while i < n:
        j, k = i + 1, i
        while j < n and s[k] <= s[j]:
            k = i if s[k] < s[j] else k + 1
            j += 1
        while i <= k:
            i += j - k
            c += 1
    return c


def stream_to_one(n: int) -> list[int]:
    """Parities along the orbit of n, up to (not including) the step taken at 1."""
    out, m = [], n
    while m != 1:
        out.append(m & 1)
        m = T(m)
    return out
```

```
70 parities to reach 1: 1101111101011011101111010011101101111110011110001010100010011100001000
Lyndon factors of that stream: 12
```

The full transcripts of the three engines are reproduced in Sections 5.1–5.9 below as they are

discussed.

## 4. GenerationalLineage

Because Sections 5.7 and 5.9 use it, we describe the engine first, as the author's own software and not as a result.

**What it is.** GenerationalLineage (repository, GenerationalLineage 1.0.0 (git 4892eaf, 2026-09-28), GPL-3.0) tracks the "lineage" of each operation in a calculation, meaning what it can be built from, in order that anything which cannot be built from the list is visible rather than silent. The author calls the project "Generational" because the word carries four meanings at once: descent, a generating set, a code generator (an iterator that holds its state), and a generation (a level or cohort). The engine and its skill are documented in the repository and in the skill file.

**The domain.** Operations are placed against a fixed domain whose floor is three primitives: **ADD** (identity 0), **SCALE** (identity 1) and **SIGN** (a single bit: a ±1 reflection). Above them are tiers: tier 1 (reflect, dilate), tier 2 (fixed sets: origin, anchor, boundary), tier 3 (counts and ratios of the layer below). An operation must land in exactly one place; one that lands nowhere is flagged as a possible new operator, and one that lands in two is taken to be mis-decomposed.

**The decomposition test.** For any named operator the engine asks, in order: is it a count or a ratio of something else (tier 3)? is it a fixed set (tier 2)? does it change length (so it needs DILATE)? does it need an added constraint to exist (a corollary, not a primitive)? Only what survives all four is a candidate primitive.

**Three kinds of wrong.** The engine keeps apart a CODE fault (the check did not run, so the claim is unjudged), a MATHS fault (both sides were measured and disagree, so the claim is false) and a METHOD error (correct code, correct maths, wrong approach, invisible to both). Section 7 uses these words.

**Factor moves.** Thirty-one toolsets share a contract (`descend` is free; `build_up` is paid or refuses with `AscentNotFree`; `verify` self-checks). This paper uses one of them, `lyndon` (unique non-increasing factorisation of a word into Lyndon words), and the engine's `ADD:SCALE:SIGN` element algebra with its recorded "Long Path" and set-membership test (the rule that decides Recamán's flip).

**Why this map.** The Collatz map was the first real computation the author ran through the engine, in August 2026, chosen as the simplest non-trivial ordering of the integers. The engine has grown since (a public 1.0.0 release, thirty-one self-verifying toolsets), so this paper re-runs the test against the current engine and does not quote the old one. **The engine and this paper share an author and an assistant; the check is not independent**, and we say so rather than imply otherwise.

**The decomposition the original engine reported** (computed, from `e_collatz_shift.py`):

| operation | tier | descends from | status |
|---|---|---|---|
| n mod 2 (branch select) | 0 | SIGN | PRIMITIVE — one bit, nothing between |
| n / 2 | 1 | SCALE -> DILATE, gain 1/2 | derived — changes length, so needs DILATE |
| 3n | 1 | SCALE -> DILATE, gain 3 | derived — changes length, so needs DILATE |

| operation | tier | descends from | status |
|---|---|---|---|
| + 1 | 0 | ADD | PRIMITIVE — identity 0; the only coupling between the two axes |
| T = the Collatz map | 3 | ADD o SCALE o SIGN | DERIVED — no new generator required |

The last row, “T = ADD ∘ SCALE ∘ SIGN”, is the statement Section 5.7 re-examines.

## 5. Results

### 5.1 T^k is affine on residue classes mod 2^k

**[ESTABLISHED:CALCULATED]**

```
collatz.affine_on_classes   [tier 1 | KNOWN (folklore)]
  descends: SCALE — a dilation of gain 3^d/2^k
  claim   : T^k restricted to n = j (mod 2^k) is the affine map (3^d n + c)/2^k, and d depends only on the CLASS,
  detail  : checked every class at every k in 1..8: 510 classes, slope = 3^d/2^k in all of them. The map is piece
affine and the pieces are indexed by Z/2^k — this IS the "recursive decomposition on a modular function".
```

### 5.2 The parity vector is a bijection; T is conjugate to the shift

**[ESTABLISHED:CALCULATED]**

```
collatz.parity_bijection   [tier 3 | KNOWN (Bernstein 1994; Bernstein-Lagarias 1996)]
  descends: SIGN — one bit per step, iterated k times
  claim   : Q_k : Z/2^k -> Z/2^k, n |-> its first k parities, is a BIJECTION
  detail  : k=1:2/2  k=2:4/4  k=4:16/16  k=8:256/256  k=12:4096/4096  k=16:65536/65536  — exhaustive, no sampling
```

**[ESTABLISHED:CALCULATED]**

```
collatz.shift_conjugacy   [tier 2 | KNOWN (Bernstein-Lagarias 1996)]
  descends: the bijection of R2 — a change of coordinates, i.e. a fixed set of the conjugating map
  claim   : Q(T(n)) = shift(Q(n)) — T is CONJUGATE to the one-sided binary shift; on Z_2 the Collatz map is chopp
  detail  : k=8:True  k=12:True  k=16:True  — verified for EVERY n in Z/2^k, not sampled. The dynamics are theref
```

### 5.3 Counting: the class tree is Pascal’s triangle

**[OURS:CALCULATED]**

```
collatz.pascal_refinement   [tier 3 | OURS (framing; the maths is elementary once stated)]
  descends: a COUNT over the classes of R1
  claim   : over the 2^k classes mod 2^k the number of odd steps d is distributed EXACTLY as C(k,d) — the branch
  detail  : exact binomial at every k in 1..12. k=8 histogram: {0: 1, 1: 8, 2: 28, 3: 56, 4: 70, 5: 56, 6: 28, 7:
```

**[OURS:CALCULATED]**

```
collatz.gaussian_from_pascal   [tier 3 | OURS (framing)]
  descends: the binomial of R4 — a ratio of counts in the limit
  claim   : log n along an orbit is a SUM of k iid log-steps whose count is Binomial(k, 1/2); the normal curve is
Laplace on R4, not an empirical observation
  detail  : k=20: sup|binomial CDF - normal CDF| = 0.00139 < 0.01 (exact binomial, no sampling). E[log n_k - log
2.8768. "Chaos in many dimensions becomes order" is this theorem and nothing more mysterious.
```

The author's reading is that the apparent chaos of the d-histogram is the binomial triangle seen in the wrong metric. We believe this to be elementary once stated, and we claim the framing at most.

### 5.4 Cycles and the denominator 2^k − 3^d

**[ESTABLISHED:CALCULATED]**

```
collatz.cycle_denominator   [tier 3 | KNOWN (standard; 2^k-3^d=1 only at (2,1) is Levi ben Gerson / Catalan, cite
  descends: a RATIO — the fixed point of the affine map of R1
  claim   : every periodic parity word of length k with d ones determines ONE rational cycle whose denominator di
integer cycle up to k=16 is {1,2}
  detail  : 8800 PRIMITIVE necklaces enumerated to k=16; positive-integer cycles found: [(2, 2, 1, 1)] — exactly
minimal representative. Ladder 2^k-3^d along the convergents of log2(3)=1.58496250: (2,1)->1, (3,2)->-
1, (5,3)->5, (8,5)->13, (13,8)->1631, (19,12)->-7153. The "near miss" is a linear form in logarithms — the same f
miss work, NOT the same problem.
```

A bounded search ($k \le 200$, $d \le 200$) finds 4 pairs with $|2^k - 3^d| = 1$: (1,0), (1,1), (2,1), (3,2). **[ESTABLISHED:CALCULATED]** for that range; that these are all such pairs with $d \ge 1$ in general is a theorem of Catalan type (Mihăilescu) which we cite and do not prove. The original text of this repository stated only the pair (2,1) for $2^k - 3^d = +1$, which is correct for $d \ge 1$ but incomplete if the absolute value is meant; we state it here with the full list.

### 5.5 Leaves, and the two directions

**[ESTABLISHED:CALCULATED]**

```
collatz.mod3_orphans   [tier 2 | KNOWN (elementary)]
  descends: a FIXED SET — the image of the odd branch, ker of nothing else
  claim   : n has an ODD predecessor iff n = 2 (mod 3); multiples of 3 are leaves of the backward tree at every s
  detail  : residues with an odd predecessor, n<300,000: [2]. (3m+1)/2 is never 0 mod 3, checked for every odd m<
```

**[OURS:CALCULATED]**

```
collatz.forward_two_to_one   [tier 3 | OURS (the measurement; the fact follows from R3)]
  descends: a COUNT of preimages — the shift of R3 is 2-to-1 by construction
  claim   : T is exactly 2-to-1 on Z_2: each forward step destroys exactly one bit. Going DOWN the tower is infor
  detail  : T is well defined as a map Z/2^k -> Z/2^(k-1); fibre sizes, uniform at every level: k=6:[2]  k=8:[2]
FAULT; the measurement was right and the assertion was wrong — a CODE fault, recorded rather than tidied away.) E
```

**[OURS:CALCULATED]**

```
collatz.backward_deficit   [tier 3 | OURS (the measurement)]
  descends: a COUNT — the complement of the fixed set of R7
  claim   : the backward tree on N has mean out-degree 4/3, not 2: going UP the tower does not propagate informat
  detail  : measured mean out-degree over n<1,000,000 = 1.333333; predicted 4/3 = 1.333333 — always 2n, plus an o
1/3 set n = 2 (mod 3). The second condition usually written here, "(2n-1)/3 must be odd", is VACUOUS and was meas
1 is odd and 3 is odd, so the quotient is odd whenever it is an integer. (First run predicted 7/6 from that phant
FAULT.) Forward destroys log 2 = 0.6931 nats per step; backward restores log(4/3) = 0.2877. The shortfall is 0.40
adic metric cannot see.
```

The reading offered is that forward T destroys one bit per step (entropy log 2), the backward tree on N restores log(4/3), and the shortfall log(3/2) is the odd branch's own gain. We believe each ingredient is elementary and we do not know whether the combined accounting appears in the literature. **OURS.**

### 5.6 The drift

**[ESTABLISHED:CALCULATED]**

```
collatz.drift_at_half   [tier 3 | KNOWN (Terras 1976; Everett 1977)]
  descends: a RATIO of the two gains of R1, weighted by the Bernoulli(1/2) measure the conjugacy of R3 preserves
  claim   : at parity measure 1/2 the per-step geometric mean is sqrt(3)/2 < 1; the drift is negative BECAUSE the
  detail  : measured mean log-step over 2,157,078 steps (odd starts < 60,000) = -0.139102; predicted log(sqrt(3)/
0.143841. Residual +0.004739 is TERMINATION BIAS — the sample stops at 1, which truncates the low tail; it is not
```

```
================================================================================
GENERATIONAL LINEAGE — T decomposed against the Two Trees domain
================================================================================
operation                  tier  descends from               status
--------------------------------------------------------------------------------
n mod 2  (branch select)   t0    SIGN                        PRIMITIVE — one bit, nothing between
n / 2                      t1    SCALE -> DILATE, gain 1/2   derived — changes length, so needs DILATE
3n                         t1    SCALE -> DILATE, gain 3     derived — changes length, so needs DILATE
+ 1                        t0    ADD                         PRIMITIVE — identity 0; the only coupling between t
T = the Collatz map        t3    ADD o SCALE o SIGN          DERIVED — no new generator required
--------------------------------------------------------------------------------
No new generator required. The Collatz map is the smallest
non-trivial composition that uses all three tier-0
irreducibles at once. ADD and SCALE have DIFFERENT identities
(0 and 1); the "+1" is the only thing coupling them, and that
coupling is the whole difficulty. Drop it and n -> 3n/2^v is a
pure SCALE tower with no conjecture in it.
================================================================================
```

### 5.7 Reading T through GenerationalLineage

**[THEORETICAL:CALCULATED]**

*What the original paper said.* "T = ADD ∘ SCALE ∘ SIGN; no new generator is required", where SIGN was taken to be the parity bit that selects a branch.

*What the current engine shows.* Written as engine elements ($x \mapsto \text{sign}\cdot\text{scale}\cdot x + \text{add}$), the even branch is SCALE(1/2) and the odd branch is ADD(1/2) after SCALE(3/2). The engine's SIGN, a ±1 factor, occurs in **neither** branch. Composing the branch elements along the true parity word reproduces T^k on every residue class mod 2^k for $k \leq 12$: 8190 classes checked, 0 mismatches, slope exactly $3^{d/2}k$. The parity bit is a *selector between two elements of the affine group*, not an application of the engine's SIGN. **The earlier sentence conflated the two and is corrected here.** What survives is the narrower statement: T is a piecewise map whose two pieces are generated by SCALE and ADD, selected by one bit. The engine's firing order, innermost first, is SIGN, SCALE, ADD, which is also the order in which the odd branch acts (select, scale by 3/2, add 1/2); firing ADD first would need the non-integer constant 1/3. This is a statement about how the author's framework names things, offered for review.

*The engine's Long-Path memory.* The engine records an operation's trajectory and the set of values already visited, the same test that decides Recamán's flip. On the orbits tried the first value to repeat is always 2, one step after the orbit reaches 1: n = 27: 70 steps to reach 1, first value to repeat is 2, at step 71; n = 97: 75 steps to reach 1, first value to repeat is 2, at step 76; n = 871: 113 steps to reach 1, first value to repeat is 2, at step 114; n = 6171: 165 steps to reach 1, first value to repeat is 2, at step 166; n = 77031: 221 steps to reach 1, first value to repeat is 2, at step 222. **[THEORETICAL:CALCULATED]** The reading is that the Collatz conjecture is equivalent to the statement that, for every n, the first repeated value in

the orbit is in the loop {1, 2}, or no value repeats (divergence). That equivalence is immediate and is not new; what the engine adds is that the test is already implemented.

### 5.8 Replication of the Bernstein–Lagarias tables

**[ESTABLISHED:CALCULATED]** for the agreements; **[OPEN:CALCULATED]** for the one disagreement.

Transcript of `python3 engine/e_collatz_bl_replication.py`:

```
Bernstein-Lagarias (1996) replication (ESTABLISHED:CALCULATED (Bernstein-Lagarias 1996, re-
computed), with ONE RECORDED DISCREPANCY WINDOW in the printed tables (see mismatch lists))
  Table 2.2 (cycle lengths on the odd residues): rows n=2..16 compared, mismatches: none
  n   cycles by length (1,2,4,...)                       order
   2  [2]                                                   1
   3  [2, 1]                                                2
   4  [4, 2]                                                2
   5  [6, 3, 1]                                             4
   6  [6, 7, 3]                                             4
   7  [8, 10, 3, 3]                                         8
   8  [14, 17, 8, 0, 3]                                     16
   9  [14, 21, 18, 4, 0, 3]                                 32
  10  [10, 35, 24, 14, 2, 0, 3]                             64
  11  [12, 40, 37, 18, 12, 2, 0, 3]                         128
  12  [16, 48, 70, 23, 16, 10, 2, 0, 3]                     256
  13  [26, 53, 79, 60, 24, 11, 10, 2, 0, 3]                 512
  14  [22, 63, 111, 98, 50, 14, 11, 10, 2, 0, 3]            1024
  15  [18, 81, 129, 153, 84, 40, 11, 11, 10, 2, 0, 3]       2048
  16  [20, 96, 179, 186, 137, 78, 31, 11, 11, 10, 2, 0, 3] 4096
  order == 2^(n-4) for every n >= 6 checked: True; every cycle length a power of two: True (checked while countir
  solenoidal property Q_n(x) = Q_(n-1)(x) mod 2^(n-1): violations 0
  odd fixed points: Q-lineage (n<=100) and the independent DFS agree: True
  Table 2.3 (n=2..100, 99 values) vs DFS: mismatches at n = [66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78]
  Table 2.4 (n=10,20,...,1000, 100 values) vs DFS: mismatches at n = [70]
  every mismatching printed entry equals the computed value at n+1 (a one-cell shift in the printed table): True
  odd fixed points at n = {10: 10, 20: 26, 30: 50, 40: 92, 50: 108, 100: 90, 200: 404, 500: 1150, 1000: 2308}
```

- The cycle table (their Table 2.2, the number of cycles of each length 2^j of Φ_n, the map on odd residues) agrees for every row we computed; the order of Q_n holds for every n from 6 to 16 and equals 2^(n−4); every cycle length is a power of two; the solenoidal property has 0 violations. These are theorems or tabulated facts in Bernstein–Lagarias; we only reproduce them.
- The odd fixed-point counts (1-cycles) are computed by two independent algorithms that agree with each other. They agree with 86 of the 99 entries of Table 2.3 and 99 of the 100 entries of Table 2.4 (n up to 1000). Odd fixed points at n = 10, 100, 200, 500, 1000: 10, 90, 404, 1150, 2308.
- **The disagreement.** The printed numbers differ from ours at n = 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78 (Table 2.3) and n = 70 (Table 2.4). In every one of them, the printed value for n equals our value for n+1: the block looks like a one-cell shift. We checked that our two algorithms agree with each other, that both agree with the printed tables at every other entry (including on both sides of the block), and that the retypeset 2022 version of the paper prints the same numbers as the 1996 preprint, so a layout or OCR error in our transcription is unlikely. We cannot exclude a fault in our code that happens to affect exactly these n, or a slip in the printed table or in the data it was built from. **We do not know which is right**; this is the first of the questions in Section 8.

### 5.9 Parity streams, controls, and a recorded fault

**[OURS:CALCULATED]**

Transcript of `python3 engine/e_collatz_gl_decomposition.py`:

```
Collatz through GenerationalLineage   [A,B: THEORETICAL:CALCULATED; C,D: OURS:CALCULATED]
  A. branch elements: even = x ↦ +1·0.5·x + 0, odd = x ↦ +1·1.5·x + 0.5; engine SIGN (+-1) in either branch: Fals
     composing the branch elements along the true parity word: 8190 classes (k <= 12), mismatches 0; engine firin
  B. Long-Path memory (n: steps to 1, first visit step, revisit step, repeated value): {27: (70, 69, 71, 2.0), 97
  C. Duval vs engine Lyndon omega on 100 random strings, disagreements: 0; rotation-minimal vs primitive words (k
  D. Lyndon factors of 115 random bits: first bit free 6.03, first bit 1 7.02, first bit 0 4.99
     mean z of the Collatz streams to 1 (100 odd starts): unmatched +1.33, first bit matched +0.95, first bit and
0.06; even starts, matched: +0.12
     METHOD error: the control was not matched on the conditioning (odd start, forced approach to 1); the excess
```

*What looked like a result.* Taking the parity stream of an orbit as a binary word, the number of factors in its Lyndon factorisation was higher for Collatz orbits than for random bit strings of the same length (8 to 10 against 5 to 7, for five orbits), while linear complexity and a grammar-compression ratio were indistinguishable from random.

*Chasing it.* (i) The engine's Lyndon tool and an independent implementation agree. (ii) Forcing the first bit to 1 raises the count for random bits themselves: a stream of 115 random bits has 6.03 factors on average, 7.02 if its first bit is 1 and 4.99 if it is 0. Every start we had used was odd, so every stream began with 1. (iii) A stream that reaches 1 must end in a nearly forced pattern (the last six bits were 001000 in 284 of 300 starts in an earlier run). (iv) Mean z-scores of the Collatz streams to 1 for odd starts: unmatched control +1.33, control matched on the first bit +0.95, control matched on the first bit and the last six bits -0.06; for even starts with matched controls +0.12. **The excess is an artefact of our control, not a property of the parity stream.** We classify it, in the engine's vocabulary, as a METHOD error (the control was not matched on the conditioning). It is recorded here because it is the kind of error a reader is likely to make, and it is *not* offered as a question.

*The sanity check that did hold.* By the conjugacy, the first k parities of a *random* n are independent fair bits; for 400-bit random n the Lyndon-factor counts match random bits with the same first bit (an earlier run: 7.02 and 5.04 against 7.04 and 5.02 for odd and even n).

## 6. What this does and does not show

It shows that three short programs reproduce the standard structure of the shortcut map and the published Bernstein–Lagarias tables (with one block of disagreement), that the map's two branches are the elements SCALE(1/2) and ADD(1/2)∘SCALE(3/2) of the affine group selected by one bit, and that one tempting statistical signature in Collatz parity streams is an artefact of mismatched controls. It does **not** bear on the conjecture, does not prove any of the cited theorems, and does not establish that the GenerationalLineage reading is useful to anyone but its author.

## 7. Faults and negative results, kept

| # | what | kind | status |
|---|---|---|---|
| 1 | Necklace filter kept rotation-minimal words, not primitive ones: one loop reported as seven (original run). Re-found with the engine's Lyndon tool: (k,d) = (4, 2): 2 rotation-minimal vs 1 primitive; (k,d) = (6, 3): 4 rotation-minimal vs 3 primitive; (k,d) = (8, 4): 10 rotation-minimal vs 8 primitive. | CODE | fixed in the original engine |
| 2 | An assertion that the fibre of T has size 4 against a correct measurement of 2 (original run) | CODE (wore a MATHS fault's label) | assertion corrected |
| 3 | A predicted backward out-degree 7/6 from a condition that does not exist: "(2n−1)/3 is odd" is vacuous, because 2n−1 and 3 are both odd | MATHS (in the prediction) | corrected; the exact value is 4/3 |
| 4 | "T = ADD ∘ SCALE ∘ SIGN" conflated a parity selector with the engine's SIGN (Section 5.7) | METHOD (labelling) | corrected in this paper; the original README is unchanged |
| 5 | A Lyndon-factor excess in Collatz parity streams: control not matched on first bit and forced tail (Section 5.9) | METHOD | resolved: artefact |
| 6 | A block of 13 entries of Bernstein–Lagarias Table 2.3 (and n = 70 of Table 2.4) disagree with two independent computations (Section 5.8) | OPEN | for the reader |

| # | what | kind | status |
|---|---|---|---|
| 7 | While gathering references, a guessed URL returned an unrelated paper; it was caught by reading the file's own title page and discarded. Four cited works could not be downloaded automatically (see `references/README.md`) | process | recorded |

## 8. Questions for the reader

1. **Table 2.3 / 2.4.** Is the printed block n = 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78 (and n = 70 in Table 2.4) in Bernstein–Lagarias a misprint or a data slip, or is our count wrong there? (Both of our algorithms are in `engine/e_collatz_bl_replication.py`; Section 5.8 gives the numbers.)
2. **The accounting in Section 5.5**: forward 2-to-1 (log 2 nats per step), backward mean out-degree 4/3 (log(4/3)), shortfall exactly log(3/2). Is this exact statement in the literature?
3. **Section 5.7**: is it reasonable to describe T as a selector between two elements of the affine group, generated by a scaling and a translation, and to keep the engine's SIGN out of it? Does this say anything beyond the observation that T is piecewise affine?
4. **Beyond n = 1000.** The second algorithm reaches n = 1000 in about two seconds. Would counts at larger n, or other statistics of Φ_n, be of interest, and are there known predictions (for instance the heuristic Bernstein–Lagarias give for the growth of the number of 1-cycles) worth testing against them?

## 9. Reproducibility

```
cd FourthAgePapers/CollatzShift
python3 engine/e_collatz_shift.py            # about 4 s: the ten relations
python3 engine/e_collatz_bl_replication.py   # about 3 s (add --full for n <= 18 cycles and the slower lineage me
python3 engine/e_collatz_gl_decomposition.py # about 7 s; needs GenerationalLineage next to this repository or GL
python3 paper/make_paper.py --check          # does this paper match a fresh build?
```

Environment of this build: Python 3.12.3, a ThinkPad X1 Carbon (6th gen, Core i7-8550U); building the paper took about 32 s. SHA-256 prefixes of the engine files used:

| file | sha256 (first 12) |
|---|---|
| `engine/e_collatz_shift.py` | `852d73561a70` |
| `engine/e_collatz_bl_replication.py` | `0749a916e353` |
| `engine/e_collatz_gl_decomposition.py` | `db01476f1905` |

The original engine's own transcript (relations and provenance as the original paper labelled them):

```
10/10 relations hold
collatz.affine_on_classes      t1   HOLDS        KNOWN (folklore)
collatz.parity_bijection       t3   HOLDS        KNOWN (Bernstein 1994; Bernstein-Lagarias 1996)
```

```
collatz.shift_conjugacy        t2   HOLDS        KNOWN (Bernstein-Lagarias 1996)
collatz.pascal_refinement      t3   HOLDS        OURS (framing; the maths is elementary once stated)
collatz.gaussian_from_pascal   t3   HOLDS        OURS (framing)
collatz.cycle_denominator      t3   HOLDS        KNOWN (standard; 2^k-3^d=1 only at (2,1) is Levi ben Gerson / Ca
collatz.mod3_orphans           t2   HOLDS        KNOWN (elementary)
collatz.forward_two_to_one     t3   HOLDS        OURS (the measurement; the fact follows from R3)
collatz.backward_deficit       t3   HOLDS        OURS (the measurement)
collatz.drift_at_half          t3   HOLDS        KNOWN (Terras 1976; Everett 1977)
collatz.affine_on_classes   [tier 1 | KNOWN (folklore)]
collatz.parity_bijection   [tier 3 | KNOWN (Bernstein 1994; Bernstein-Lagarias 1996)]
collatz.shift_conjugacy   [tier 2 | KNOWN (Bernstein-Lagarias 1996)]
collatz.pascal_refinement   [tier 3 | OURS (framing; the maths is elementary once stated)]
collatz.gaussian_from_pascal   [tier 3 | OURS (framing)]
collatz.cycle_denominator   [tier 3 | KNOWN (standard; 2^k-3^d=1 only at (2,1) is Levi ben Gerson / Catalan, cite
collatz.mod3_orphans   [tier 2 | KNOWN (elementary)]
collatz.forward_two_to_one   [tier 3 | OURS (the measurement; the fact follows from R3)]
collatz.backward_deficit   [tier 3 | OURS (the measurement)]
collatz.drift_at_half   [tier 3 | KNOWN (Terras 1976; Everett 1977)]
```

## References

**The Collatz conjecture itself (the 3x+1 problem) is built-against and carries no tag; every numbered item below is `Post-Hoc:` except item 7.** Copies of the works we could download are in `references/` (and in ContextPlease); the ledger of what was and was not downloaded is `references/INDEX.md`.


1. `Post-Hoc:` R. Terras, A stopping time problem on the positive integers, *Acta Arith.* **30** (1976), 241–252. (not downloaded)
2. `Post-Hoc:` C. J. Everett, Iteration of the number theoretic function f(2n) = n, f(2n+1) = 3n+2, *Adv. Math.* **25** (1977), 42–45. (not downloaded)
3. `Post-Hoc:` J. C. Lagarias, The 3x+1 problem and its generalizations, *Amer. Math. Monthly* **92** (1985), 3–23. (not downloaded)
4. `Post-Hoc:` D. J. Bernstein, A non-iterative 2-adic statement of the 3x+1 conjecture, *Proc. Amer. Math. Soc.* **121** (1994), 405–408. (not downloaded; the publisher's site returned a bot-check page)
5. `Post-Hoc:` D. J. Bernstein, J. C. Lagarias, The 3x+1 conjugacy map, *Canad. J. Math.* **48** (1996), 1154–1169. (`references/Bernstein_Lagarias_1996_retypeset.pdf`, preprint copy also included)
6. `Post-Hoc:` J. C. Lagarias, The 3x+1 problem: an annotated bibliography (1963–1999), arXiv:math/0309224v13. (`references/Lagarias_annotated_bibliography_arXiv_math0309224.pdf`; read only for its existence and scope)
7. (built-against; the author's own software, no tag) C. M. Allison, The Generational Lineage Engine, version 1.0.0 (2026), https://github.com/michaelrendier/GenerationalLineage.


(Citation details for items 1–4 are from memory or from publisher listings and should be checked before this draft is circulated further. The paper's own code was executed; the cited papers' proofs were not re-derived.)

---

## Appendix A. What in ContextPlease was used, and what ContextPlease is

**What ContextPlease is.** ContextPlease is a versioned, human-readable archive of the working context of the author's projects, kept so that a model, or a person, who has never seen the

work can come up to speed with historical access. It holds the standing rules and notation, a per-repository current-state file, a dated log of what happened, the *skills* (reusable working methods the assistant is given), every research scratchpad with its scripts and outputs, copies of context primers and wiki pages, and, since 2026-10-14, a ledger of every third-party work downloaded. It is deliberately "manual transmission": nothing in it is loaded automatically; the reader chooses what to read and in what order. A new model should start with `README.md` (rules, traps, layout), then `claude/.clauderc_context_1` (what is true now) and `claude/.clauderc_context_2` (what happened), then the skills it is pointed at.

**What was used for this paper.**

| in ContextPlease | role in this paper |
|---|---|
| `README.md` | the standing rules in force: no renormalisation, no fitted parameters; all failures stay in the code and the data |
| `claude/skills/cs-paper-code-conventions/SKILL.md` | how code appears in a paper: verbatim, numbered, executed-cold listings for short mechanisms; transcripts for sessions; paths for long files; no formula standing in for a loop |
| `claude/skills/generational-lineage/SKILL.md` | the decomposition test, the tiers, the three kinds of wrong, the label discipline (Sections 4, 5.7, 7) |
| `claude/scratchpad/2026-10-14_collatz_gl_rerun/` | the exploratory scripts and outputs behind Sections 5.7–5.9 before they were folded into `engine/` |
| `claude/references/INDEX.md` and `references/CollatzShift/` | the literature ledger and the downloaded copies (the Literature Download Protocol) |
| `claude/.clauderc_citations` | the queue of citations by repository |
| `claude/.clauderc_canonical_maths` (block `@RCCM_CONFIDENCE_TIERS`) | the label tiers, including the `:CALCULATED` suffix |
| `claude/hooks/clauderc_primer.py` | the start-of-session map of the files above (documentation copy) |

Not in ContextPlease: the assistant's local memory notes (for example the rule that every download of another person's work is recorded and mirrored beside the paper, and that outbound requests carry a browser-style user agent). Their content is summarised in the ledger and the primer where it matters.

*This appendix is a standing section of every paper the author now writes with an assistant.*